

Smart Farming Roadmap — Part 2

Smart Farming (SIH 25099) — Stage 10 Onward Implementation Plan
Backend · Frontend · Mobile — detailed build plan continuing from Stage 9 (Configuration Layer)
Includes: Depth Additions (Farm Context Engine, Safety Rule Engine, Expert Escalation, and more)

1. Where You Are Right Now

Stages 1–9 of the v2 build order are implemented: OpenCV preprocessing, crop identification, the Decision Engine, per-crop disease classifiers, severity estimation, the pest-detection stub, the recommendation service, the context-object pipeline orchestrator, and the YAML configuration layer. The entire "AI brain" already runs end-to-end from a Python script, driven purely by `pipeline.py` and `config.yaml` — no web server involved yet.

Stage	Name	Status
1	OpenCV Preprocessing Service	Done
2	Crop Identification Service (EfficientNet-B0)	Done
3	Decision Engine (crop → disease model routing)	Done
4	Disease Classification Services (per-crop)	Done
5	Severity Estimation Service	Done
6	Pest Detection Service (YOLOv8, stretch)	Done / stubbed
7	Recommendation Service (LLM + weather)	Done
8	Context Object + Pipeline Orchestrator	Done
9	Configuration Layer (config.yaml)	Done — you are here

Everything from here on is **plumbing and delivery**: exposing the pipeline over HTTP, logging every prediction, and building the three surfaces a farmer or judge will actually touch — a web dashboard, a mobile app, and (behind the scenes) a monitoring/retraining loop.

2. Remaining Build Order (Stage 10 → Stage 18)

Keep the same bottom-up discipline as before: get each layer working in isolation, then wire it up. Backend endpoints and logging come first because both the web dashboard and the mobile app depend on a stable

API contract — build the contract once, consume it twice.

Stage	Name	Owner	Depends on
10	FastAPI Endpoints (/predict, /auth, /history, /feedback)	Backend	Stage 8, 9
11	Logging Layer (full-chain prediction + feedback logs)	Backend	Stage 10
12	Auth & Farmer/Farm Profile Layer	Backend	Stage 10
13	React Web Dashboard	Frontend	Stage 10, 12
14	Mobile App (Flutter)	Mobile	Stage 10, 12
15	Deployment (Docker, Nginx, CI/CD)	Backend / DevOps	Stage 10–14
16	MLOps Loop (monitoring, drift, retraining trigger)	Backend / Data Science	Stage 11
17	Testing & QA (unit, integration, load)	Cross-cutting	Stage 10–16
18	Demo Packaging & Documentation	Cross-cutting	Stage 17

Stages 13 and 14 can be built in parallel by different teammates once Stage 10's API contract is frozen.

3. Backend Implementation Plan

3.1 Stage 10 — FastAPI Endpoints

main.py stops being a thin demo wrapper and becomes a real routed application. Split routes into routers by concern (predictions, auth, feedback, admin) instead of one flat file.

Method & Path	Purpose	Auth?
POST /auth/register	Create farmer account (name, phone, location, language)	No
POST /auth/login	Issue JWT access + refresh token	No
POST /predict	Upload leaf image → runs full pipeline → returns result	Yes
GET /predictions/{id}	Fetch one stored prediction + recommendation	Yes
GET /history	Paginated list of a farmer's past predictions	Yes
POST /feedback	Farmer rates a prediction (correct / incorrect + note)	Yes
GET /weather	Current weather for farmer's saved location	Yes
GET /crops	List of supported crops (for mobile/web dropdowns)	No
GET /health	Liveness/readiness probe for orchestration	No
GET /admin/metrics	Aggregate accuracy & confidence stats (Stage 16)	Admin

Route skeleton

```
backend/
■■■ main.py                # creates app, mounts routers, CORS, middleware
■■■ api/
■   ■■■ deps.py           # get_current_user, get_db, get_pipeline
■   ■■■ routes_predict.py # /predict, /predictions/{id}
■   ■■■ routes_auth.py    # /auth/register, /auth/login
■   ■■■ routes_feedback.py # /feedback
■   ■■■ routes_history.py # /history
■   ■■■ routes_admin.py   # /admin/metrics

# api/routes_predict.py
@router.post("/predict")
async def predict(file: UploadFile, user=Depends(get_current_user)):
    context = create_context(file, user)
    result = run_pipeline(context)      # Stage 8 orchestrator, unchanged
    log_prediction(result)              # Stage 11
    return serialize(result)
```

Nothing about the pipeline itself changes — **run_pipeline()** from Stage 8 is called as-is. FastAPI's job is authentication, request/response shaping, and error handling (return a clear 422 when quality checks fail, not a raw 500).

3.2 Stage 11 — Logging Layer

Every request should leave a permanent, queryable trail from raw upload to farmer feedback. This is the table that eventually powers both the MLOps retraining loop and the admin dashboard.

Table	Key columns
users	id, phone, name, language, created_at
farms	id, user_id, location, crop_history (json)
images	id, user_id, raw_path, processed_path, quality_score, blur, leaf_detected
predictions	id, image_id, crop, crop_conf, disease, disease_conf, model_used, severity_pct, created_at
recommendations	id, prediction_id, fertilizer, pesticide, irrigation, prevention_tips
feedback	id, prediction_id, is_correct, farmer_note, created_at

```
# database/logging.py
def log_prediction(context: dict) -> None:
    with session_scope() as db:
        image = Image(**context["image"], user_id=context["user"]["user_id"])
        db.add(image); db.flush()
        pred = Prediction(image_id=image.id, crop=context["crop"]["label"],
                          crop_conf=context["crop"]["confidence"],
                          disease=context["disease"]["label"],
                          disease_conf=context["disease"]["confidence"],
                          model_used=context["disease"]["model_used"],
                          severity_pct=context["severity"]["percent"])
        db.add(pred); db.flush()
        db.add(Recommendation(prediction_id=pred.id, **context["recommendation"]))
```

Pair this with structured application logging (Python **logging** configured to emit JSON lines) so ops issues — slow inference, model load failures, low-confidence spikes — show up in logs independently of the database.

3.3 Stage 12 — Auth & Farmer/Farm Profile Layer

- JWT access + refresh tokens (short-lived access token, longer refresh token stored securely on device).
- Farmer profile: name, phone/OTP or email, preferred language (drives the LLM recommendation language), farm location.
- Farm/crop history: which crops a farmer grows, used later to bias the recommendation prompt and pre-fill the mobile app's crop picker.
- Password hashing with bcrypt/passlib if email+password is used; OTP-over-SMS is usually a better fit for a farmer-facing product than passwords.

3.4 Deployment (Stage 15)

```
# docker-compose.yml (services)
services:
  api:          # FastAPI app, gunicorn+uvicorn workers
  postgres:     # structured data
  minio:        # image blob storage (S3-compatible)
  nginx:        # reverse proxy + TLS termination + static file cache
  redis:        # optional: job queue for long-running inference, rate limiting
```

- Nginx sits in front of FastAPI: terminates TLS, proxies /api/* to uvicorn, serves the built React app as static files.
- Model weights (.pth files) are mounted as a volume, not baked into the image — keeps image rebuilds fast when you swap a model version.
- CI/CD (GitHub Actions): on push → run pytest → build Docker image → push to registry → (optional) deploy to a VM/Render/Railway for the demo.
- Keep a staging config.yaml and a production config.yaml — same code, different thresholds/paths, matching the Stage 9 design.

4. Frontend Implementation Plan (React Web Dashboard)

The web dashboard's job is fast: judges and admins use it to browse predictions, not necessarily to upload from the field — that's what the mobile app is for. Keep it a thin, fast consumer of the API you already froze in Stage 10.

4.1 Pages / Routes

Route	Purpose
/login, /register	Auth screens, same API as mobile
/upload	Drag-and-drop leaf photo → calls POST /predict → shows a loading state
/result/:id	Crop, disease, severity, Grad-CAM heatmap overlay, recommendation card
/history	Paginated table/grid of past predictions with filters (crop, disease, date)
/profile	Farmer/farm details, crop history, language preference
/admin	Aggregate metrics: accuracy over time, confidence distribution, drift flags (Stage 16)

4.2 Component & State Architecture

```
frontend/src/  
■■■ api/           # thin fetch wrappers per endpoint (predict.ts, auth.ts, history.ts)  
■■■ components/  
■   ■■■ UploadDropzone.tsx  
■   ■■■ ResultCard.tsx      # crop + disease + confidence badges  
■   ■■■ SeverityGauge.tsx   # radial gauge for severity %  
■   ■■■ GradCamOverlay.tsx  # heatmap image overlay  
■   ■■■ RecommendationPanel.tsx  
■   ■■■ HistoryTable.tsx  
■■■ pages/         # one file per route above  
■■■ context/AuthContext.tsx # holds JWT, current user  
■■■ hooks/usePredict.ts    # React Query mutation wrapping POST /predict  
■■■ App.tsx
```

- Data fetching: React Query (TanStack Query) for caching, retries, and loading/error states on every API call — avoids hand-rolled useEffect spaghetti.
- Auth: JWT stored in memory + httpOnly refresh cookie if the backend supports it; AuthContext gates protected routes.
- Result screen is the centerpiece: show the original photo, the Grad-CAM heatmap side by side, confidence as a percentage badge (color-coded by threshold from config.yaml), and the LLM recommendation as a readable card, not raw JSON.
- Language: the context object already carries a language field — read it from the farmer profile and request recommendations in that language; keep UI copy in a small i18n dictionary (English/Gujarati/Hindi) from day one rather than retrofitting it.

5. Mobile Implementation Plan (Flutter)

This is where most real farmers will actually interact with the system — from a field, often with weak connectivity. The mobile app is not a shrunk version of the dashboard; camera capture and offline resilience are first-class requirements, not stretch goals.

5.1 Why a Dedicated Mobile App

- Camera-first UX: a farmer photographs a leaf directly, with in-app guidance (fill the frame, avoid shadows) instead of picking a file.
- Low/no connectivity in rural areas: predictions must be queueable offline and synced later.
- Push notifications: alert the farmer when an async recommendation (e.g. weather-dependent advice) is ready.

5.2 Screens

Screen	Purpose
Splash / Onboarding	Language selection, permissions (camera, location, notifications)
Login / OTP	Phone-based auth, matches backend Stage 12
Home	Big "Scan a Leaf" button, recent predictions, weather strip
Camera Capture	Live camera with a leaf-alignment guide overlay, retake option
Upload / Processing	Progress indicator while the image uploads and the pipeline runs
Result	Crop, disease, severity, recommendation — voice playback option for low-literacy users
History	Offline-cached list of past scans, synced predictions marked vs. pending ones
Profile / Farm Details	Farmer info, crop history, saved farm location
Settings	Language, notification preferences, data-usage mode (compress images on mobile data)

5.3 Offline-First Architecture

1. Farmer takes photo → compressed locally (image + metadata saved to local SQLite queue)
 2. If online: upload immediately → POST /predict → result cached locally, queue entry removed
 3. If offline: entry stays in queue with status = "pending_sync"
 4. Connectivity listener detects reconnection → background sync worker drains the queue in order
 5. Result arrives → local notification fires → History screen updates the entry's status
- State/data layer: Riverpod or Provider for app state; sqflite (SQLite) for the offline queue and cached history.
 - Image handling: compress and resize client-side before upload (target similar resolution to the 224×224 the model expects, or a slightly larger safe size) to save data and speed up sync on 2G/3G.
 - Networking: dio for HTTP with retry/backoff interceptors; reuse the exact same API contract as the web dashboard (Stage 10) — no separate mobile-only backend logic.

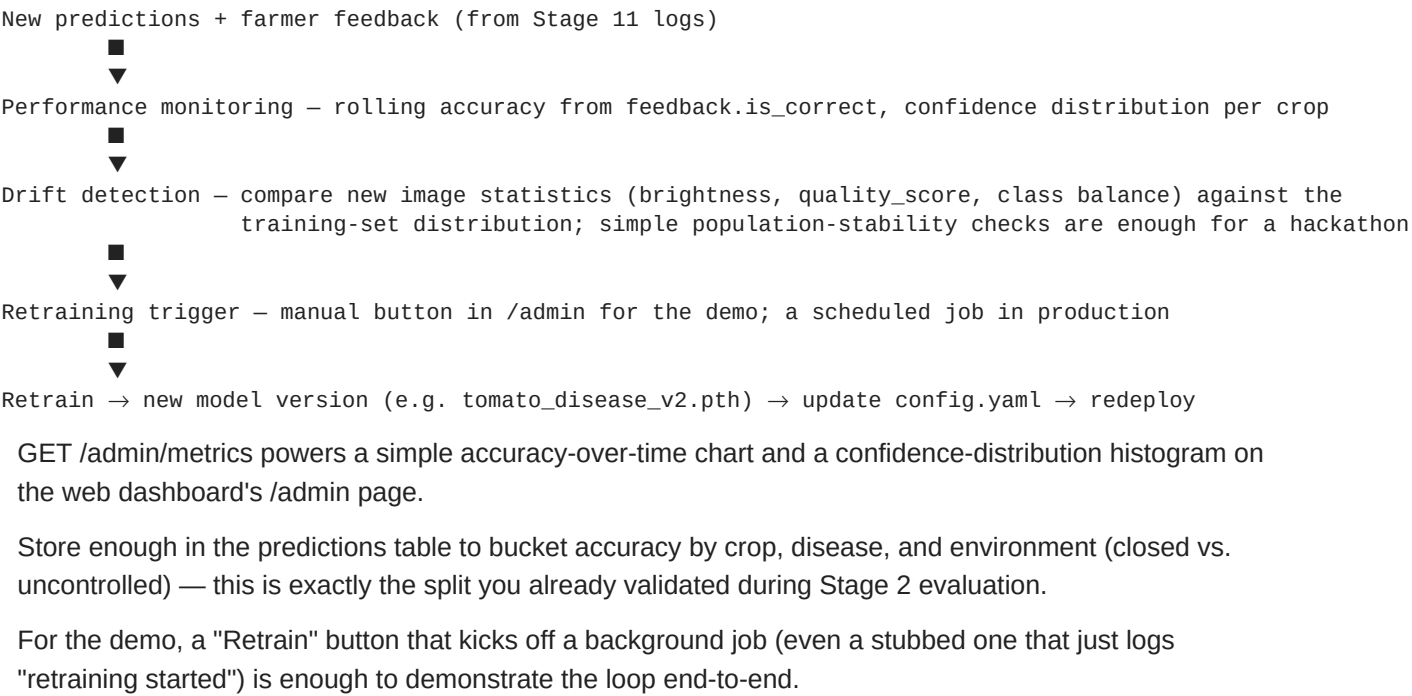
- Push notifications: Firebase Cloud Messaging for "your recommendation is ready" and periodic crop-care reminders.
- Accessibility: many farmers may have low literacy — lean on icons, color-coded severity, and optional text-to-speech for the recommendation text.

5.4 Folder Structure

```
mobile/lib/
■■■ main.dart
■■■ screens/      # one folder per screen above
■■■ services/
■   ■■■ api_service.dart      # mirrors frontend/src/api wrappers
■   ■■■ sync_service.dart     # offline queue drain logic
■   ■■■ notification_service.dart
■■■ models/        # Prediction, Recommendation, Farmer, SyncQueueItem
■■■ providers/     # Riverpod providers: auth, predictions, connectivity
■■■ widgets/       # ResultCard, SeverityGauge, CameraGuideOverlay (shared visual language with web)
```

6. MLOps Loop Implementation (Stage 16)

This is the stage judges consistently reward even when it's only partially built — showing the monitoring/retraining loop is real, not a slide, matters more than making it fully automatic.



7. Testing & QA (Stage 17)

Layer	Tooling	Focus
Backend unit	pytest	Each service function in isolation (preprocess, identify_crop, etc.)
Backend integration	pytest + TestClient	Full /predict flow against a test DB and sample images
Load	locust	Concurrent /predict calls — inference latency under load
Frontend	Vitest / React Testing Library	Component rendering, API-hook mocking
Mobile	flutter_test	Widget tests for camera flow, offline-queue logic

8. Combined Folder Structure

```
Smart-Farming/
├── backend/
│   ├── main.py
│   ├── pipeline.py           # unchanged from Stage 8
│   ├── config.yaml          # unchanged from Stage 9
│   ├── api/                  # NEW – Stage 10 routers
│   ├── database/
│   │   ├── models.py
│   │   ├── logging.py       # NEW – Stage 11
│   │   ├── services/        # unchanged (preprocessing, crop_identifier, ...)
│   │   ├── models/          # .pth weights
│   │   └── utils/
│   └── frontend/            # NEW – Stage 13, React (Vite)
│       ├── src/ (api, components, pages, context, hooks)
│       └── mobile/          # NEW – Stage 14, Flutter
│           ├── lib/ (screens, services, models, providers, widgets)
│           └── deploy/       # NEW – Stage 15
├── docker-compose.yml
├── nginx.conf
├── datasets/
└── notebooks/
```

9. Full Tech Stack Reference (Updated)

Layer	Tools
Backend API	FastAPI, Uvicorn/Gunicorn, Pydantic, PyJWT, passlib
Data & Storage	PostgreSQL, MinIO/S3, SQLAlchemy, PyYAML
Frontend	React (Vite), TypeScript, React Query, Tailwind/shadcn
Mobile	Flutter, Riverpod, sqflite, dio, Firebase Cloud Messaging
AI Models	EfficientNet-B0/B2, ConvNeXt, YOLOv8, Llama 3 / Gemini
Deployment	Docker, Docker Compose, Nginx, GitHub Actions
Testing	pytest, locust, Vitest, flutter_test

10. Suggested Milestones

Milestone	Covers
M1	Stage 10 + 11: FastAPI endpoints wired to the existing pipeline, predictions logged to Postgres
M2	Stage 12: Auth working end-to-end (register/login/JWT), farmer profile stored
M3	Stage 13: Web dashboard hits real API — upload, result, history pages functional
M4	Stage 14: Mobile app — camera capture, upload, result screen, basic offline queue

Milestone	Covers
M5	Stage 15: Dockerized deployment, demo-able over a public URL
M6	Stage 16 + 17: Admin metrics page, retrain button stub, core tests passing
M7	Stage 18: Demo script, README, architecture diagram polished for submission

Stage 10 and 11 are the critical path — every other surface (web, mobile, admin) is a consumer of that API. Freeze the request/response shapes from Stage 10 as early as possible so frontend and mobile can be built in parallel against a mocked API before the real backend is fully wired up.

11. Depth Additions — Strengthening the Core Pipeline

These are deliberately **not** new product surfaces (no marketplace, logistics, IoT, or scheme-finder modules). Each one deepens a stage you are already building — Recommendation (7), Logging (11), Auth/Profile (12), or Mobile (14) — so the project stays a focused AI diagnosis product rather than sprawling into an unrelated agri-platform.

Addition	Slots into	What it changes
Farm Context Engine	Stage 7 — Recommendation	Prompt includes zone, farm history, weather, growth stage — not just crop+disease
Rule Engine + Safety Validator	Stage 7 — Recommendation	Deterministic checks wrap the LLM output before it reaches the farmer
Expert escalation on low confidence	Stage 12 — Auth/Profile	Low-confidence predictions get a "pending expert review" status instead of a shown answer
Explainable AI panel	Stage 13 — Web Dashboard	Grad-CAM heatmap + a short evidence list alongside the diagnosis, not just a percentage
Multi-plot / farm zones + history	Stage 11 — Logging	Predictions grouped by a farmer-defined plot; trend view over time on data already logged
Offline-first + regional-language voice	Stage 14 — Mobile	Already-scoped mobile work; explicitly protected from being cut for time
Weather-triggered proactive alerts	Stage 7 — Recommendation	Existing weather API call also checks disease-risk thresholds and can push an alert

11.1 Farm Context Engine

Extend the context object's inputs to the recommendation prompt beyond crop + disease + severity:

```
prompt_context = {
    "crop": context["crop"]["label"],
    "disease": context["disease"]["label"],
    "severity": context["severity"]["percent"],
    "zone": context["farm"]["zone_id"],
    "growth_stage": context["farm"]["growth_stage"],
    "weather": context["weather"],
    "recent_actions": context["farm"]["recent_actions"], # e.g. "fungicide applied 5 days ago"
    "prior_disease_in_zone": context["farm"]["zone_history"],
}
```

This is additive to the existing prompt-construction step in Stage 7 — no new service, just a richer input drawn from fields you are already storing in the farm/zone profile (Stage 12) and logs (Stage 11).

11.2 Rule Engine + Safety Validator

```
IF disease_confidence < config.thresholds.disease_confidence:
    status = "pending_expert_review" # do not show unverified advice

IF recommendation.missing(["dosage", "crop", "disease"]):
    recommendation = "Insufficient information – please consult an agricultural expert."

IF severity > config.thresholds.high_severity AND pest_detected:
```

```
create_alert(priority="HIGH")
```

Runs as a small deterministic step between the LLM call and the API response in Stage 7 — a handful of if/else rules reading from the same config.yaml thresholds you already defined in Stage 9.

11.3 Expert Escalation on Low Confidence

Confidence	Outcome
High ($\geq$ disease_confidence threshold)	Auto-shown to farmer as normal
Low (below threshold)	Status = pending_expert_review; farmer sees "forwarded for expert review"

Requires one new status value on the predictions table (Stage 11) and one new role — Expert — in the auth layer (Stage 12), who can view pending cases and approve/modify the diagnosis via an admin route.

11.4 Explainable AI Panel

On the result screen (web Stage 13, mobile Stage 14), pair the diagnosis with the Grad-CAM heatmap you already planned and a short plain-language evidence list (e.g. "Brown lesions detected", "Leaf discoloration detected") instead of a bare confidence percentage. No new model — this reads directly from Grad-CAM output already produced during evaluation (Stage 3 of the original notebooks).

11.5 Multi-Plot / Farm Zones + History Timeline

```
farms
■■■ id, user_id, location
zones
■■■ id, farm_id, label ("North Plot"), crop, area
predictions
■■■ ...existing columns..., zone_id ← NEW foreign key
```

A single foreign key added to the Stage 11 predictions table lets the history page (Stage 13/14) group and chart results per plot instead of one flat list — pure UI work on data you are already logging.

11.6 Offline-First + Regional-Language Voice

This is not new scope — it is Stage 14's offline queue (Section 5.3) and language field (already in the context object) protected as must-have, not trimmed if the mobile build runs short on time. Voice output is text-to-speech over the same recommendation text already generated in Stage 7, in the farmer's saved language.

11.7 Weather-Triggered Proactive Alerts

```
IF weather.humidity > crop_disease_risk[crop][disease].humidity_threshold
  AND weather.temperature in crop_disease_risk[crop][disease].temp_range:
  push_alert(farmer, "High-risk conditions for {disease} – inspect your {crop} today")
```

Reuses the weather API call already made in Stage 7's recommendation step; the only addition is a small static risk-threshold table per crop/disease and a push notification (web push or the Stage 14 mobile notification service) instead of only surfacing weather inside a recommendation.